This is the comprehensive Project Description and Development Roadmap for **Meda Pass**.

This document serves as the blueprint for developers and stakeholders. It incorporates the "Fayda" (National ID) authentication offloading, the 15-minute buffer logic, and the HTMX polling strategy.

Project Name: Meda Pass

Mission: To digitize sports infrastructure booking in Ethiopia, making access to fields seamless via Telebirr while providing government and private operators with transparent management tools.

1. Executive Summary

Meda Pass is a dual-interface platform.

1. **User Side (Telebirr Mini App):** A lightweight interface for citizens to find nearby fields, log in via their National ID (FAN), and book slots using Telebirr.
2. **Admin & Ops Side (Web Portal):** A hierarchical dashboard for Woreda/Subcity supervisors to manage assets and for on-site Gatekeepers to validate bookings via QR codes.

---

2. Technical Stack (Speed-to-Market Optimized)

We will use a "Monolith" architecture. This avoids the complexity of managing separate API and Frontend codebases, allowing for rapid iteration.

- **Backend Framework: Laravel 11 (PHP)**.
  - *Reason:* Best-in-class queue management, simple scheduling, and rapid admin panel generation.
- **Database: PostgreSQL**.
  - *Features used:* ACID compliance for bookings, PostGIS (future-proofing for maps), JSONB for operating hours.
- **Admin Interface: FilamentPHP**.
  - *Reason:* Auto-generates the Woreda/Subcity dashboards and Operator CRUDs, saving weeks of UI work.
- **Mini App Frontend: Laravel Blade + HTMX + Alpine.js** .
  - *Reason:* You requested HTMX. It is perfect here. It allows the Mini App to feel like a Native SPA (no page reloads) without the heavy download size of React/Vue. It handles the "Polling" and "Blinking UI" effortlessly.
- **Real-time & Caching: Redis**.
  - *Usage:* Managing booking locks (concurrency), caching slot availability, and processing queues.
- **Payment Gateway: Telebirr SuperApp SDK / API**.

---

3. Functional Specifications

A. Authentication (The "Fayda" Logic)

- **Production Flow:**
  1. User enters **FAN** (Fayda ID Number).
  2. System calls National ID API.
  3. **Success:** API returns `{first_name, last_name, dob, photo, phone}` .
  4. System checks DB:
     - *If User Exists:* Log them in.
     - *If New:* Auto-register user using data from National ID and log them in.
- **Development/Testing Flow:**
  - Since we lack the license currently, we will build a **Mock Auth Service**.
  - Devs can log in using standard Email/Password, but the database structure will strictly enforce the `national_id` column for the future switch.

B. The Booking Engine (Concurrency & Buffers)

1. **Slot Generation:**
   - Logic: `Field Open Time` + `1 Hour Play` + `15 Min Buffer` .
   - *Example:* 8:00-9:00 (Play) -> 9:00-9:15 (Buffer/Switch) -> 9:15-10:15 (Play).
2. **Live Availability (WebSocket + HTMX Polling):**
   - **Primary:** Laravel Reverb (WebSockets) pushes updates. When User A books a slot, User B's screen turns that slot Gray instantly.
   - **Fallback (HTMX):** If the socket drops, HTMX triggers a background poll every 5 seconds (`hx-trigger="every 5s"`) to refresh the slot status.
   - **Visuals:** A "Blinking Green" dot on the UI indicates live data is active.
3. **Locking (Redis):**
   - When a user clicks "Book", a **5-minute Redis Lock** is placed on that slot. Other users see "On Hold".

C. Cancellation & Rescheduling

- **Rule:** Cancellations allowed > 2 hours before start time.
- **Refunds:** No cash reversals.
- **Store Credit:** System creates a `Credit` record. Next time the user books, the system checks `if (user_credit > 0)` .

D. Gatekeeper/Operator Module

- **Access:** Web Portal (Mobile View).
- **Auth:** Username/Password (Created by Woreda Admin).
- **Action:**
    1. User shows QR Code.
    2. Operator scans.
    3. System validates: `Field == Correct`, `Time == Now`, `Status == Paid`.
    4. System marks booking as `CHECKED_IN`.

---

4. Database Schema (Postgres)

```
-- USERS (Populated via National ID API in Prod)
CREATE TABLE users (
    id BIGSERIAL PRIMARY KEY,
    national_id VARCHAR(50) UNIQUE NOT NULL, -- The FAN
    first_name VARCHAR(100),
    last_name VARCHAR(100),
    phone VARCHAR(20) UNIQUE, -- Retrieved from ID or Manual input
    email VARCHAR(150) NULL,
    dob DATE NULL,
    store_credit DECIMAL(10, 2) DEFAULT 0.00, -- For cancellations
    role VARCHAR(20) DEFAULT 'USER', -- USER, OPERATOR, SUPER_ADMIN
    created_at TIMESTAMP DEFAULT NOW()
);

-- HIERARCHY
CREATE TABLE locations (
    id SERIAL PRIMARY KEY,
    region VARCHAR(100), -- Addis Ababa
    subcity VARCHAR(100), -- Bole
    woreda VARCHAR(100) -- Woreda 03
);

-- FIELDS
CREATE TABLE fields (
    id SERIAL PRIMARY KEY,
    location_id INT REFERENCES locations(id),
    operator_user_id BIGINT REFERENCES users(id), -- The Gatekeeper
    name VARCHAR(150),
    type VARCHAR(50), -- FOOTBALL, BASKETBALL
    price_per_hour DECIMAL(10, 2),
    status VARCHAR(20) DEFAULT 'ACTIVE', -- ACTIVE, MAINTENANCE
    operating_hours JSONB, -- {"start": "06:00", "end": "22:00"}
    created_at TIMESTAMP DEFAULT NOW()
);

-- BOOKINGS
CREATE TABLE bookings (
    id BIGUUID PRIMARY KEY, -- UUID to prevent ID guessing
    user_id BIGINT REFERENCES users(id),
    field_id INT REFERENCES fields(id),
    start_time TIMESTAMP NOT NULL,
    end_time TIMESTAMP NOT NULL, -- Includes the play time only, not buffer
    status VARCHAR(20), -- PENDING_PAYMENT, PAID, CANCELLED, COMPLETED
    payment_ref VARCHAR(100), -- Telebirr Transaction ID
    qr_hash VARCHAR(255), -- Secure string for QR generation
    checked_in_at TIMESTAMP NULL,
    created_at TIMESTAMP DEFAULT NOW()
);
```

---

5. Development Roadmap (Phased)

Phase 1: The Core & Mock Auth (Weeks 1-3)

Goal: A working booking flow with dummy data.

- **Backend:** Setup Laravel, Postgres, and the DB Schema.
- **Admin (Filament):** Build the dashboard to Create Locations, Fields, and assign Operators.
- **Auth (Mock):** Create a Login page that asks for "FAN". For now, it accepts any 10-digit number and logs you in as a Test User.
- **Booking Engine:** Implement the 15-minute buffer logic and Redis locking.

Phase 2: The Mini App & Telebirr (Weeks 4-6)

Goal: Payment integration and User UI.

- **Frontend (HTMX):** Build the mobile views using Blade/HTMX.
  - *View 1:* Search/Filter (Dropdowns for Subcity).
  - *View 2:* Slot Selector (Grid view with Live Status).
  - *View 3:* Checkout.
- **Telebirr:** Integrate the H5/Macle payment SDK. Handle the "Callback" webhook to confirm payment and generate the QR code.
- **Notification:** Setup Email queues for booking confirmation.

Phase 3: Gatekeeper & National ID Prep (Weeks 7-8)

Goal: Operations and Real-world prep.

- **Operator View:** Create a simple mobile page for operators to see "Today's Schedule" and a "Scan QR" button (using `html5-qrcode.js`).
- **National ID:** If the license is ready, swap the Mock Auth for the real API call. If not, refine the Mock to strictly match the API spec for a later switch.
- **Testing:** Load testing the WebSocket/Polling fallback to ensure it works on slow 3G networks.

---

6. Immediate Next Steps for Developer

1. **Initialize Laravel Project:** Install Laravel 11, Filament, and Breeze (for base auth scaffolding).
2. **Migration:** Run the SQL schema provided above.
3. **HTMX Setup:** Install `laravel-htmx` or simply add the CDN to the main layout file.
4. **Mock Data:** Seed the database with 1 Region (Addis), 1 Subcity (Bole), and 2 Fields to start coding the slot logic.